

INTERPRETACIÓN SEMÁNTICA DE SOLICITUDES WEB EN CAPA DE APLICACIÓN

Alumno: Cozzi, Alejandro Luis (Leg. N° 27826374)

PROFESOR: Salgado, Ariel

TALLER: Introducción a la Metodología de Investigación – Maestría en Ciencia de Datos

BUENOS AIRES
SEGUNDO CUATRIMESTRE - 2026

1. Introducción

1.1. Contexto

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

1.2. Justificación del estudio

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

1.3. Alcances del trabajo y limitaciones

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

2. Estado de la cuestión

Mejorar los niveles de detección de ataques en WAF a partir de los datos, es un problema constante y recurrente dado que la cantidad y creatividad de los ataques se sostiene en el tiempo. Las dinámicas elementales desde el punto de vista técnico para quienes no estén familiarizados con el problema de la seguridad y la acción del WAF son trabajadas en [1] donde se realiza una exploración metodológica acerca de la arquitectura necesaria para detectar y bloquear amenazas de tipo SQLI; si bien menciona los aportes que podrían hacer las técnicas de deep learning en esa capa, no incluye ninguna en su propuesta. Sí aporta algunas nociones metodológicas que podrían ayudar en la etapa de experimentación y, sobre todo, al momento de comparar con los filtros heurísticos. En cambio [2] realiza una aproximación muy similar a la que pretende plantear este trabajo, aunque sobre presupuestos metodológicos diferentes. Si bien parte de arquitecturas transformers para la comprensión semántica de cada solicitud web, apunta al fine-tuning a partir de RoBERTa, antes que a la extracción de embeddings crudos para su posterior procesamiento. No buscan generalizar sobre un conjunto de solicitudes maliciosas per se, sino que proponen realizar un entrenamiento de modelos singulares para cada aplicación web. Es probable que el entrenamiento singular tenga mucho sentido a la hora de detectar anomalías, aunque no queda del todo claro en ese trabajo cómo

se realizaría el etiquetado ya que en los experimentos se utilizan datasets ya etiquetados como CSIC 2010 y DRUPAL. En una segunda etapa, luego del fine-tuning de RoBERTa, utilizan OneClassSVM un clasificador binario para detectar los ataques en los datasets, y lo comparan con los resultados TPR y FPR usando ModSec como WAF. La potencia de RoBERTa también fue utilizada en [3] donde los autores experimentaron con un ensamble bastante más complejo pero con algunos parámetros similares. Su trabajo también utiliza herramientas transformers, RoBERTa en particular y el dataset de CSIC 2010 para pruebas, además del dataset ATRDF 2023. Su propuesta se inspira en el framework GAN de forma muy original: identifican tokens reservados en el dataset original y los enmascaran con un token de máscara (literalment <MASK>), luego utilizan RoBERTa para generar el reemplazo esperable de esa fracción del código, lo cual entrega un dataset paralelo, distinto del original. A continuación utilizan nuevamente RoBERTa para comparar ambos, y calculan la similaridad del coseno de cada palabra para determinar anomalías. Por último, entrenan un clasificador de Random Forest para predecir y discriminar entre «tráfico normal» y «posible ataque». Otra característica en común de los dos trabajos mencionados anteriormente es que ambos obtienen como resultado esa clasificación binaria. En cambio en [4] los autores apuntan a entregar ambos tipos de clasificación, una binaria (que discrimine ataque / legítimo) y una multiclase, clasificando hasta 8 clases en total (incluyendo al tráfico legítimo). Partiendo de la extracción de embeddings de la solicitud http, proceden a añadirle ruido usando el algoritmo Da-Sana. Para construir los embeddings utilizaron Keras con un padding posterior para normalizar los largos (es decir recortando las líneas que exceden el largo determinado y completando con ceros las que no llegan a él) usando los datasets CSIC-2010 y ECML/PKDD. Un desarrollo más amplio y complejo se encuentra en [5]. En este trabajo se presenta un proceso que acuña su propio nombre: DeepPTSD una sigla que describe básicamente el proceso propuesto, de Deep Learning Payload Traffic Detection Method Transfer Semi-Supervised Detection. Se trata de un ensamble bastante sofisticado que se entrena con datasets clasificados públicos y aspira a detectar líneas maliciosas en un WAF. A grandes rasgos el ensamble sigue un proceso similar a los demás, aunque con variantes en cada paso, por ejemplo los embeddings se extraen con Word2Vec luego se pre-entrena un modelo con esa información usando redes convolucionales. Con los resultados se realiza una aumentación de los datos que sigue dos caminos: «keyword mining» que parte de un dataset de payloads y aplica una ecuación tf-idf para crear una biblioteca de payloads y luego una «payload perturbation» que reemplaza ciertos tokens que detecta similares a los de la biblioteca en la línea a analizar.

Los trabajos mencionados parecen coincidir en un problema central: las reglas de WAF son eficientes para detectar un alto porcentaje de solicitudes maliciosas, pero son incapaces de detectar algunos tipos de ofuscación. Además, por su propia naturaleza, surgen como reacción a la detección de vulnerabilidades de tipo «zero-day» y esa reacción abre una ventana de oportunidad para el ciberdelito. Es por eso que se busca en todos una solución al problema a partir de los datos, con distintas estrategias que en definitiva resultan en una acción inmediata sobre una solicitud web, en la capa de WAF. Esta solución no implica un reemplazo del sistema de reglas tradicional, sino un complemento que mejore las métricas de detección. En cualquier caso, eso significa básicamente aumentar el TPR y reducir el FPR con distintas estrategias específicas de la ciencia de datos, en un margen de tiempo poco considerable.

3. Definición del problema

La seguridad de los activos digitales es un problema a nivel global que dista de poder resolverse de forma taxativa. Hay muchos enfoques al respecto, en este trabajo se toma como referencia el modelo OSI [6] definido por la organización internacional ISO para la seguridad en las comunicaciones computacionales. Más allá del problema global, este trabajo se focalizará en tres problemas dependientes de éste, los cuales se intentarán explicar a continuación. El modelo OSI define 7 capas abstractas de seguridad, una de las cuales, la capa 3 se ocupa de la seguridad de red. Es en esa capa donde se colocan los cortafuegos de aplicación (WAF por Web Application Firewall), un filtro lógico que media entre una aplicación e internet y se ocupa, entre otras funciones, de definir si una solicitud es legítima o no para proceder a su bloqueo o

redireccionamiento hacia el servicio. Esa definición no es sencilla por varios motivos y la definición que se toma no siempre es la correcta. La OWASP Foundation (por sus siglas en inglés Open Web Application Security Project) reconoce 78 distintos tipos de ataques, aunque la cifra está en constante movimiento. Cada uno de ellos no llega al WAF con una etiqueta de ‘malicioso’ en el encabezado, sino que lo habitual es que se recurra a distintas técnicas de ofuscación para evitar la detección, lo cual constituye el primer problema. Y esa detección, recurre principalmente a expresiones regulares (regex) que buscan patrones dentro de la solicitud que podrían ser signo de un ataque de los mencionados. OWASP provee de forma abierta cerca de 300 reglas para ese fin a través del programa CRS (Owasp Core Rule Set) que se pueden implementar en cualquier WAF. Owasp desarrolló un WAF de código abierto, llamado ModSecurity (de ahora en más Modsec) al cual se le pueden definir esas reglas, con distintos niveles de paranoia. A mayores niveles de paranoia, más cantidad de falsos positivos, a menor, más posibilidades de bypass, lo cual constituye el segundo problema. El tercer problema es procedimental: Supongamos que un atacante descubre una vulnerabilidad en un sistema (momento llamado zero-day), los responsables de ese servicio vulnerado detectan la intrusión, lo reportan, Owasp u otro organismo produce una regla para ese exploit, publican la regla y se reconfigura el WAF con la actualización. Este pipeline produce necesariamente una ventana de oportunidad muy importante, que en 2025 promedió los 241 días de vulnerabilidad.

4. Hipótesis

A partir de un conjunto de líneas de log de WAF con tráfico web, etiquetados con una clasificación de ciberseguridad (normal o malicioso) es posible generar nuevas etiquetas para líneas desconocidas, sin necesidad del filtro heurístico del WAF sino con un algoritmo que determine la naturaleza de la solicitud, realizando un trabajo igual o superior al que realiza un WAF. Esta hipótesis parte de ciertos supuestos que vale la pena mencionar para comprender los alcances y límites esperados. El primero es que es poco probable detectar un zero-day: por su naturaleza es algo nuevo, desconocido hasta el momento y que si bien puede guardar relación con algunas de las técnicas de ofuscación utilizadas en otros ataques, rompe con lo conocido hasta el momento. Por otro lado, la propia clasificación de la que se parte para ofrecer una solución a partir de los datos, depende necesariamente de líneas que hayan sido clasificadas previamente, con lo cual, superada la etapa de pruebas con un dataset etiquetado, la solución encontraría problemas serios para avanzar sobre un sistema productivo, dado que dependería de un factor externo (como el WAF u otro modelo entrenado) etiquetando los datos para entrenamiento. En cualquier caso la solución no se propone como un reemplazo al WAF y sus reglas, sino como un complemento adyacente al WAF que pueda ofrecer una solución híbrida para aumentar la seguridad.

5. Objetivos

5.1. Objetivo General

Clasificar correctamente un conjunto de líneas de log asociadas a tráfico web normal o malicioso, con un ratio igual o superior al que realiza actualmente un WAF.

5.2. Objetivos Específicos

- Extraer de cada línea de log, las características semánticas particulares que nos aporten información sobre la intencionalidad de la solicitud.
- Identificar las características de la línea que más peso tienen dentro de la definición de la etiqueta.
- Generar un pipeline eficiente que permita procesar solicitudes en menos de 3 segundos.

6. Solución Propuesta

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetur odio sem sed wisi.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

7. Metodología

Se espera lograr un pipeline de datos que permita la experimentación y prueba. Para el entorno global se utilizará Kubernetes como infraestructura de soporte, que provea contenedores necesarios para las tareas basados en la filosofía DevSecOps.

a) Ingesta

El software de soporte inicial sería una base de datos Postgres (dentro de un contenedor) que podría usar psycopg para ingestar u otro contenedor con Python y SQLAlchemy para la interacción. Por la misma naturaleza de las líneas (muchas con caracteres especiales y binarios) es necesario insertar línea por línea con una codificación amplia como «latin-1» con un control flexible de errores. Para trabajos de análisis se podría ingestar idealmente en una tabla Postgres que contenga un id, la línea en cuestión y su/s label/s.

b) Limpieza y parseo

Esta etapa es compleja por varios motivos y las decisiones que se tomen en ésta pueden cambiar de raíz los resultados finales. En principio no se puede realizar una normalización como se haría con otros sets de datos, ya que eliminar caracteres especiales o aplanar el case (llevar todo a mayúsculas o a minúsculas), nos haría perder información elemental para nuestro objetivo. Por otro lado, la presencia, dentro del propio contenido de los datos, de símbolos que habitualmente se emplean como separadores de campos incrementa la complejidad del proceso de parseo. Caracteres como la coma (,), el punto y coma (;), el numeral (#) o incluso caracteres de control como el salto de línea (\n), el retorno de carro (\r) y la tabulación (\t) son utilizados con frecuencia por los atacantes para alterar el comportamiento esperado de los mecanismos de procesamiento de datos.

En consecuencia, es necesario analizar individualmente cada conjunto de datos para identificar con precisión la posición de cada variable y asignarla correctamente a los campos de la estructura tabular que serán utilizados en las etapas posteriores del procesamiento.

c) Selección de variables e ingeniería de características

Se debería realizar un concatenado de el método más el cuerpo de la solicitud (body) que incluiría la uri, el payload y el user-agent. Estos elementos están en discusión y la definición requiere un análisis más profundo sobre el dataset en particular. Para la ingeniería de características se podrían hacer varios cálculos sobre la línea en sí, incluyendo: entropía de Shannon, puerto destino, cantidad de caracteres especiales, cantidad de palabras clave, ratio de caracteres especiales sobre el total, largo de la línea, etc.

d) Extracción de embeddings

Sobre las variables seleccionadas y concatenadas, debería realizarse la extracción de embeddings, a partir de distintas librerías de transformers para probar cuál aporta mejores resultados. En este punto es necesario tomar otra decisión metodológica, relacionada al largo de la línea a tomar: si truncar los resultados sobre el largo de la mayoría y perder parte de los datos (usando un percentil de largo de 95 por ejemplo) o utilizar el máximo perjudicando la velocidad y el consumo de recursos. En este punto no tendría sentido mantener los datos en el motor SQL; debería exportarse a un archivo que puedan consumir los algoritmos de entrenamiento como podría ser parquet o archivos de matrices.

e) Entrenamiento

Con los datos exportados, probar el entrenamiento con variantes de boosting y SVM cambiando parámetros y porcentajes de entrenamiento y prueba sobre el dataset.

f) Prueba, pruebas cruzadas y recolección de métricas

El modelo entrenado, además de probarse con los datos que se apartaron para pruebas del mismo dataset, podrían testearse de forma cruzada con datos de otro dataset para ver la flexibilidad del modelo.

7.1. Técnicas / Herramientas

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

8. Referencias-Bibliografía

Ejemplo de cómo citar una fuente en el cuerpo del texto: [1] o [2].

- [1] A. F. Hassan, «IMPROVING WEB APPLICATION SECURITY VIA FIREWALL-BASED DETECTION AND PREVENTION OF SQL INJECTION ATTACKS», *Journal of Digital Security and Forensics*, vol. 3, n.º 1, págs. 67-74, 2026. dirección: <https://doi.org/10.29121/digisecforensics.v3.i1.2026.104>
- [2] N. Montes, G. Betarte, R. Martínez y A. Pardo, «Web application attacks detection using deep learning», en *Iberoamerican Congress on Pattern Recognition*, Springer, 2021, págs. 227-236. dirección: https://link.springer.com/chapter/10.1007/978-3-030-93420-0_22
- [3] U. Aharon, R. Marbel, R. Dubin, A. Dvir y C. Hajaj, «RoBERTa-Augmented Synthesis for Detecting Malicious API Requests», *arXiv preprint arXiv:2405.11258*, 2024. dirección: <https://arxiv.org/abs/2405.11258>
- [4] H. Karacan y M. Sevri, «A novel data augmentation technique and deep learning model for web application security», *IEEE Access*, vol. 9, págs. 150 781-150 797, 2021. dirección: https://www.academia.edu/108290354/A_Novel_Data_Augmentation_Technique_and_Deep_Learning_Model_for_Web_Application_Security
- [5] T. Chen et al., «A payload based malicious http traffic detection method using transfer semi-supervised learning», *Applied Sciences*, vol. 11, n.º 16, pág. 7188, 2021. dirección: <https://doi.org/10.3390/app11167188>

- [6] J. D. Day y H. Zimmermann, «The OSI reference model», *Proceedings of the IEEE*, vol. 71, n.º 12, págs. 1334-1340, 1983. dirección: <https://ieeexplore.ieee.org/abstract/document/1457043>